



AI for Theological Inquiry

Lecture 1 — The course, agentic AI, and your toolkit

Huihuo Zheng
Wheaton College

September 2, 2026

huihuo.zheng@wheaton.edu

Course description

Originally: *Knowledge-Based AI for Theological Inquiry and Text Analysis*

Exploration of the applications of artificial intelligence to **theological texts** and **multi-perspective question answering**. Topics include **retrieval-augmented generation**, **vector databases**, **knowledge graphs**, **AI agents**, and **evaluation of attribution, faithfulness, and bias** in generated responses.

Students participate in a **collaborative research project**: the design, construction, and evaluation of a knowledge-based AI system for multi-perspective theological question answering.



Limited enrollment · faculty approval required.

Today: one lecture, one hands-on setup



Part 1 — Lecture

- What this course is
- The syllabus & the 4 phases
- The mentored-research structure
- A first look at **agentic AI**



Part 2 — Hands-on

- Install VS Code
- Install a coding agent:
Claude Code or **Codex**
- Feel the agent loop yourself

Goal: you leave with the plan, the intuition, and a working environment.

Three pillars, and they build on each other

Agentic AI

The system that **plans**, uses **tools**, and **acts** over many steps.

RAG

How it stays **grounded in real texts** instead of inventing them.

Evaluation

How you **know** whether any of it is trustworthy.

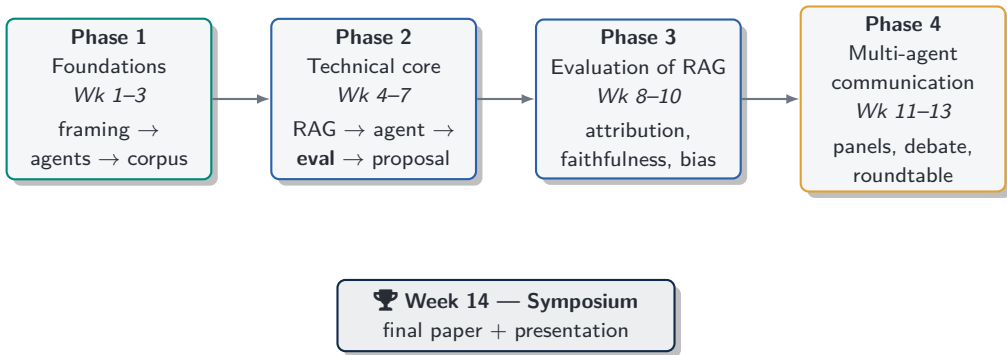
Ground it. **Make it act.** **Prove it works.**

Why evaluation gets equal weight

⚠ In theology, a fabricated scripture reference is a scholarly *and* pastoral failure.

A language model is fluent and confident — and will happily produce a plausible, *non-existent* verse. So we do not trust its claims; we **check** them. That is why we spend as much time on evaluation as on building.

14 weeks, four phases, one project through-line



Each week: ~40 min concept · ~40 min lab · ~20 min project touchpoint.

How you are graded

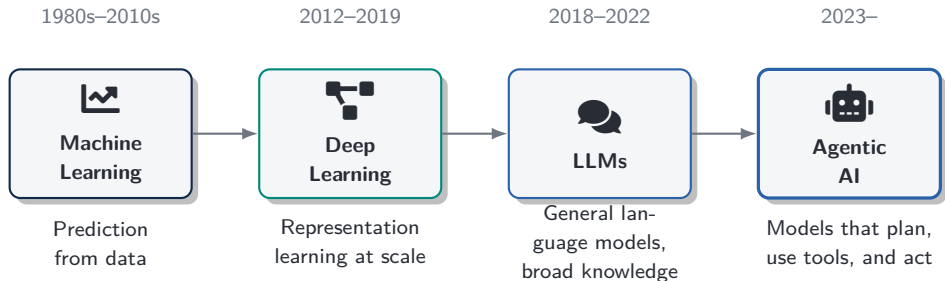
Component	Weight
Participation	40%
Paper	30%
Presentation	30%

What “mentored research” means for you

- **One original project all term** — a researchable theological question + an agentic method + an **evaluation plan**.
- **Continuous mentoring** — weekly 20-min touchpoints, so you never drift far before a check-in.
- **Low-code by default** — you build from provided notebook templates; an optional **light-code (Python)** path exists. *No one writes an agent or eval harness from scratch.*
- **One corpus, one agent** — RAG (Wk 4) grounds the agent (Wk 5); evaluation (Wk 6) judges both. Same texts all term.

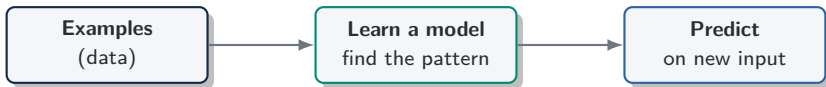
Ground it. Make it act. Prove it works — on *your* question.

A short history of AI: from predicting to acting



The shift is from **predicting** to **generating** to **acting**.

Machine learning: patterns from data, not hand-written rules



Supervised

Learn from **labeled** examples to predict a label — e.g. “is this passage about grace or law?”

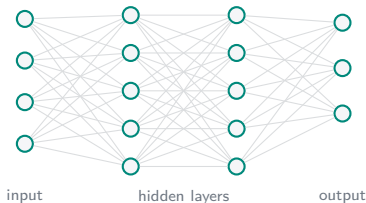


Unsupervised

Find **structure** without labels — e.g. cluster passages by theme, or find the most similar verse.

The machine *finds* the rule from data; we do not write it by hand.

Deep learning: neural networks that learn their own features



- **Many layers**, each building a richer representation of the input.
- Learns the useful **features itself** — no hand-engineering.
- Took off with **big data** + **GPUs** (2012 onward).

Different data, different architecture: CNNs for images, transformers for text.

Representation learning at scale — the engine under modern AI.

Large language models: predict the next word, at scale



Trained on massive text to predict the next token — repeated billions of times.

✓ What it gives you

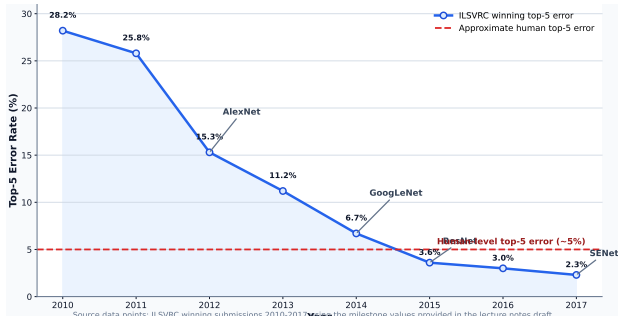
Fluent, general language and broad knowledge; it can summarize, translate, draft, and reason over text.

⚠ The catch

Not a database. It can state a **false** "fact" — a fabricated verse — with full confidence.

This is exactly why we add **RAG** (grounding) and **evaluation**.

Why *now*? Compute + data + a scalable architecture



- **2012 — AlexNet.** A deep CNN on GPUs cut ImageNet error from $>25\%$ to 15.3%. Known ideas + GPU compute + big data.
- **2017 — Transformer.** *Attention Is All You Need* replaced recurrence with self-attention $\Rightarrow$ parallel, scalable training. The basis for GPT, BERT, Llama, Claude, Gemini.
- **2022 — ChatGPT.** Made the leap public; the systems story is the same — coordinate compute, memory, and data.

This is the wave your project rides.

LLM, agent, agentic system — not the same thing

LLM

- Primarily **responds** to a prompt
- Generates text, code, or analysis
- Usually stops after one answer
- Does not manage execution itself

Examples: GPT-5, Claude

Agent

- Wraps an LLM with **tools + memory + control logic**
- Can plan, act, observe, and continue
- Executes a task instead of only answering
- Uses tools and recovers from failures

Examples: GPT-5 → Codex, Claude → Claude Code

 **Agentic system:** the operational stack around one or more agents — tools, memory, permissions, policies, scheduling, user interaction, and recovery.

What turns an LLM into an *agent*?

An LLM is a **next-token predictor**: fluent, not a database — and it will confidently **hallucinate** a verse. Add four things:

Tools

Call a retriever, a calculator, a code runner, the web.

Memory

Carry state across steps — what it found, what it decided.



Plan–act– observe

Decompose, act, *look at the result*, decide the next step.

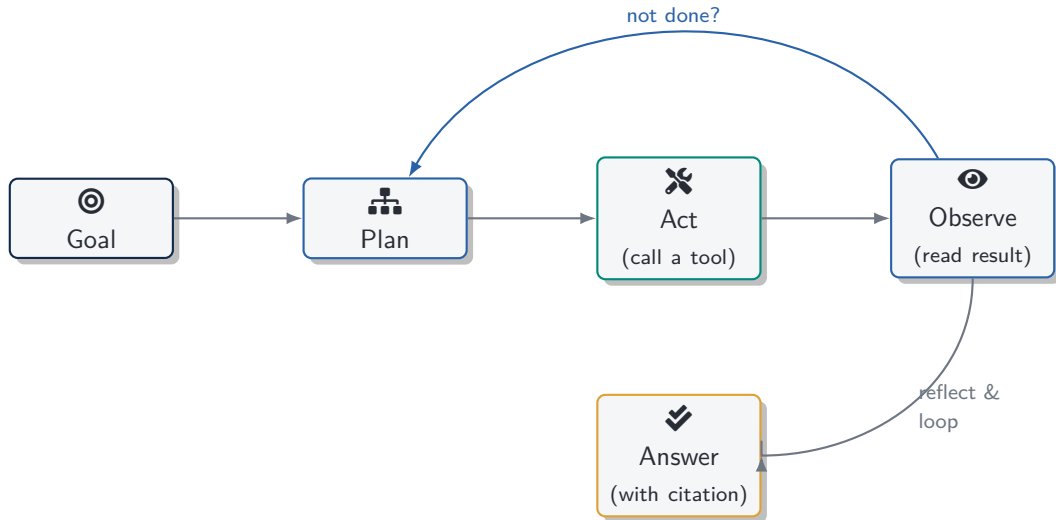


Autonomy

Run many steps toward a goal without a human at each one.

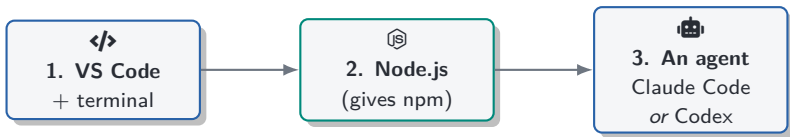
An LLM answers in one shot. An **agent** runs the loop.

The loop: plan → act → observe → repeat



Part 2 — Hands-on: install the workshop

i **Claude Code** (Anthropic) and **Codex** (OpenAI) are terminal-based coding agents that live inside your editor — the *agentic AI* we just met, pointed at files. **You only need one.**



Use the instructor-provided access. Never paste a personal API key onto a shared machine.

Install: VS Code → Node → one coding agent

✓ Prerequisites

1. Install **VS Code** — code.visualstudio.com
2. Open a terminal: Terminal → New Terminal
3. Install **Node.js (LTS)** — nodejs.org, then verify:

```
node --version  
npm --version
```

🤖 Pick *one* agent

Claude Code (Anthropic)

```
npm install -g @anthropic-ai/claude-code  
claude
```

Codex (OpenAI)

```
npm install -g @openai/codex  
codex
```

First run opens a browser sign-in. Use the instructor-provided access.

Feel the loop on purpose — then you are set for the term

➤ In your agent (claude or codex), type a **two-step** request:

“Make a folder `notes/`, then inside it create `week1.md` with a bullet list of the three pillars of this course.”

Watch it **decompose** the task, **act**, **check** the result, and **continue**.

That is the agentic loop — the same idea we aim at theological corpora in Week 4.

Assignment for next week

1. Apply for ALCF access

Apply for an ALCF account at `accounts.alcf.anl.gov`. During the application, request to be added to the **DLIO** project.

2. Reading assignment

- **Lewis et al.** (2020), *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* — read the abstract and introduction.
- **Ji et al.** (2023), *Survey of Hallucination in Natural Language Generation* — read §1.
- **Write a one-page report** on one of the two papers.

The PDFs are provided with the course materials.